Métodos de Conjunto — Parte II

Florestas Aleatórias (Random Forests), Validação OOB e Importância de Atributos

Prof. Rodrigo

Instituto Federal de São Paulo - IFSP Campus Guarulhos
Disciplina de Aprendizado de Máquina

Semestre Letivo

Objetivos de Aprendizagem — Parte II

Ao final desta segunda parte, o aluno será capaz de:

1. **Explicar a Injeção de Aleatoriedade em Florestas Aleatórias:** Compreender por que o Bagging trava no piso $\rho\sigma^2$ e como o sorteio de $m = \sqrt{d}$ atributos por nó descorrelaciona as árvores ($\rho \to 0$).
2. **Dominar o Algoritmo de Indução e Extra-Trees:** Analisar a convergência sem *overfitting* pela Lei dos Grandes Números e o sorteio de limiares em *Extra-Trees*.
3. **Formalizar a Validação Gratuita Out-of-Bag (OOB Score):** Equacionar a agregação restrita aos subconjuntos $\mathcal{S}_i$ de árvores que não viram a amostra no treino.
4. **Diferenciar MDI de Permutação OOB:** Identificar o viés da Redução Média de Impureza em variáveis de alta cardinalidade e aplicar a Importância por Permutação.
5. **Executar a Resolução Manual e Configurar o Scikit-Learn:** Rastrear passo a passo a indução, votação e acurácia OOB em um dataset finito ($N = 5$) e parametrizar o `RandomForestClassifier`.

Recapitulação da Parte I: O Limite da Variância no Bagging

De Onde Partimos?

Na Parte I, demonstramos que a variância da predição média de B árvores treinadas sobre amostras *Bootstrap* com variância individual σ^2 e correlação par a par ρ é dada por:

$$\mathrm{Var}(\hat{f}_{\mathrm{bag}}) = \underbrace{\rho\sigma^2}_{\text{Piso Fixo}} + \underbrace{\frac{1-\rho}{B}\sigma^2}_{\text{Cai a zero com } B\to\infty}$$

A Pergunta Central desta Aula

Aumentar o número de árvores B zera apenas o segundo termo, deixando o erro estagnado no piso $\rho\sigma^2$. Por que a correlação ρ entre árvores é tão alta no Bagging simples e como podemos forçar $\rho \to \mathbf{0}$?

O Gargalo da Correlação (ρ) no Bagging Simples

Por que o Bagging atinge um Gargalo?

Se o conjunto de dados contiver um ou dois atributos **muito fortes** (ex: tamanho da pétala em Iris):

- **Saturação do Nó Raiz:** Quase todas as B árvores do Bagging escolherão esse mesmo atributo na raiz.
- **Árvores Estruturalmente Idênticas:** As árvores geradas terão aparências e cortes quase iguais.
- **Alta Correlação ($\rho \approx 0{,}8$):** Isso impede que a variância do conjunto diminua, pois a curva trava no piso $\rho\sigma^2$.

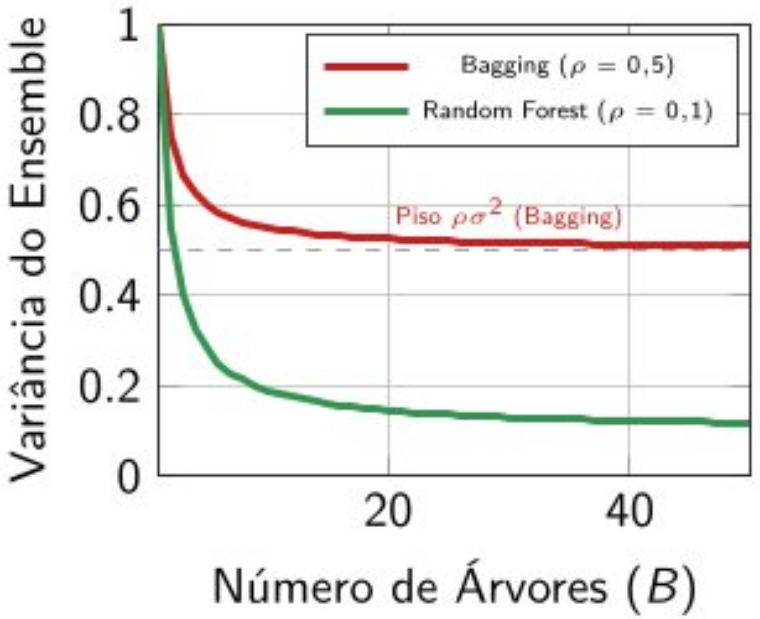


Figura: A redução da correlação ρ em Florestas Aleatórias rebaixa o piso de variância.

Injeção de Aleatoriedade em Florestas Aleatórias

A Solução de Leo Breiman (2001)

Para romper a correlação entre as árvores, o algoritmo de **Florestas Aleatórias (*Random Forests*)** introduz uma regra de **subamostragem de atributos** (*Feature Subsampling*) a cada nó de cada árvore.

Regra de Seleção de Atributos em Florestas Aleatórias

Ao dividir qualquer nó de qualquer árvore, em vez de buscar o melhor corte entre **todos os d atributos**, o algoritmo sorteia aleatoriamente um subconjunto de $m < d$ **atributos** candidatos. A busca pelo melhor corte é restrita a esses m atributos.

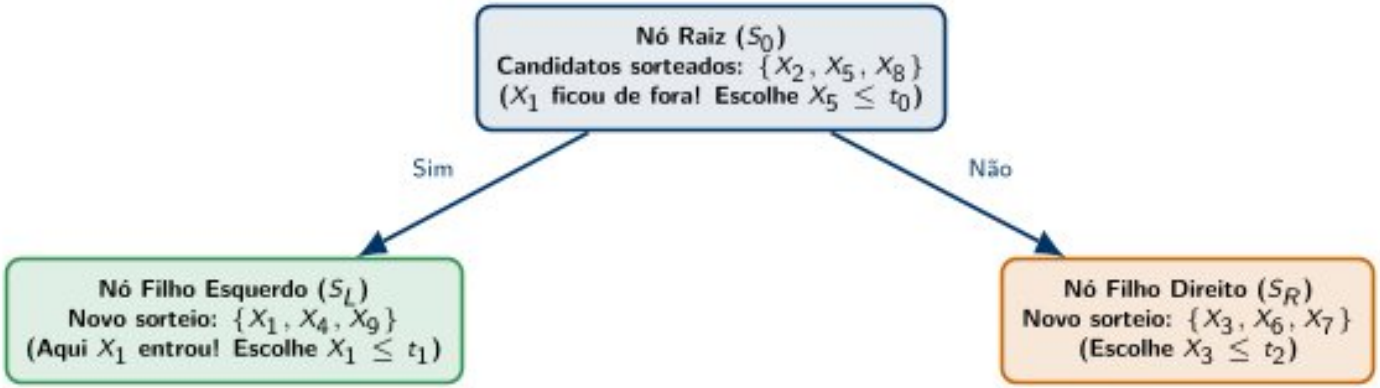
- **Para Classificação (Padrão Recomendado):** $m = \lfloor \sqrt{d} \rfloor$.
- **Para Regressão (Padrão Recomendado):** $m = \lfloor d/3 \rfloor$.

Efeito Matemático

Ao proibir a consulta aos atributos dominantes em vários nós, força-se a árvore a testar variáveis secundárias. Isso gera árvores altamente descorrelacionadas ($\rho \to 0$), colapsando o piso de variância!

Visualização do Sorteio de Atributos Nó a Nó ($d = 9 \implies m = \sqrt{9} = 3$)

Suponha que X_1 seja um atributo **dominante**. Veja o que ocorre quando sorteamos apenas $m = 3$ atributos candidatos em cada nó:



Por que isso descorrelaciona as árvores?

Na raiz desta árvore, X_1 tem probabilidade $\frac{d-m}{d} = \frac{9-3}{9} = \mathbf{66{,}7\%}$ de **não participar** da disputa! Assim, dois terços das árvores da floresta terão raízes baseadas em atributos alternativos (X_5, X_2, etc.), explorando interações secundárias ricas nos dados.

Algoritmo Completo de Indução de Floresta Aleatória

Algoritmo: Treinamento de Random Forest (Breiman, 2001)

1. **Para** $b = 1$ até B **faça:**
   1. Sorteie uma amostra bootstrap $\mathcal{D}^{(b)}$ de tamanho N com reposição a partir de $\mathcal{D}$.
   2. Treine uma árvore T_b sobre $\mathcal{D}^{(b)}$ expandindo-a recursivamente para cada nó até atingir o tamanho mínimo de folha:
      - Sorteie aleatoriamente m atributos sem reposição dos d atributos originais.
      - Escolha o melhor atributo $X_j \in m$ e o melhor ponto de corte t_k que maximiza o Ganho de Impureza (Gini / Entropia / MSE).
      - Divida o nó em dois nós filhos e repita.
2. **Retorne** o comitê de árvores $\{T_1, T_2, \ldots, T_B\}$.

Ausência de Overfitting ao Aumentar o Número de Árvores (B)

Desmistificando o Aumento de B

Diferente de redes neurais ou modelos de Boosting, **adicionar mais árvores B a uma Floresta Aleatória NUNCA causa overfitting**.

Prova pela Lei dos Grandes Números

Pela Lei dos Grandes Números, quando $B \to \infty$, a predição da floresta converge quase certamente para o seu limite de esperança matemática:

$$\lim_{B \to \infty} \hat{f}_{\text{RF}}^{(B)}(\boldsymbol{x}) = E_{\Theta}\left[T(\boldsymbol{x}; \Theta)\right]$$

onde Θ é a variável aleatória que representa o sorteio do bootstrap e dos atributos.

Regra Prática

Aumentar B apenas estabiliza a predição do ensemble (reduzindo a parcela $\frac{1-\rho}{B}\sigma^2$ até zero). O único custo de utilizar $B = 500$ em vez de $B = 100$ é o tempo de processamento e memória RAM.

Variante: Árvores Extremamente Aleatórias (Extra-Trees)

Extra-Trees

As ***Extremely Randomized Trees*** (`ExtraTreesClassifier`) levam a injeção de aleatoriedade a um nível ainda mais extremo:

- Para cada um dos m atributos sorteados no nó, o algoritmo sorteia **limiares de corte (t_k) totalmente aleatórios** dentro do intervalo $[X_{\min}, X_{\max}]$ daquele atributo no nó.
- Em seguida, escolhe o melhor corte apenas entre esses m limiares aleatórios.

Vantagens Práticas do Extra-Trees

- **Velocidade Computacional:** Não perde tempo ordenando valores e testando todos os $N-1$ pontos médios para buscar o corte ótimo.
- **Maior Redução de Variância:** Descorrelaciona ainda mais o comitê (ρ menor).
- **Troca Viés-Variância:** Apresenta um viés ligeiramente maior que a Random Forest, mas menor variância.

Avaliação Out-of-Bag (OOB) como Validação Gratuita

Algoritmo de Computação do Erro OOB (OOB Score)

Para cada amostra $\boldsymbol{x}^{(i)} \in \mathcal{D}$ do dataset original:

1. Identifique o subconjunto $\mathcal{S}_i$ de árvores para as quais $\boldsymbol{x}^{(i)}$ foi **Out-of-Bag** (não participou do treino — cerca de $0{,}368B$ árvores).
2. Obtenha a predição OOB agregando apenas as respostas das árvores de $\mathcal{S}_i$:

$$\hat{y}_{\text{OOB}}^{(i)} = \text{Agregação}_{T_b \in \mathcal{S}_i} \left(T_b(\boldsymbol{x}^{(i)}) \right)$$

3. O erro OOB global é a perda média computada sobre todas as N amostras:

$$\text{Erro}_{\text{OOB}} = \frac{1}{N} \sum_{i=1}^{N} L\left(y^{(i)}, \hat{y}_{\text{OOB}}^{(i)} \right)$$

Resultado: O OOB Score fornece uma estimativa imparcial equivalente à Validação Cruzada K-Fold sem precisar separar um conjunto de validação ou re-treinar modelos.

Importância de Atributos via MDI (*Gini Importance*)

Como não é possível visualizar 500 árvores ao mesmo tempo, medimos a relevância de cada atributo X_j somando o **quanto ele limpou os nós** em toda a floresta:

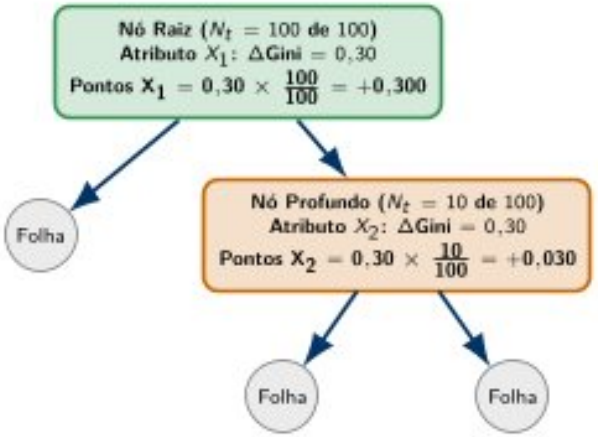
A Ideia da MDI (Redução Média de Impureza)

Cada vez que um atributo X_j divide um nó t, ele ganha uma pontuação proporcional à **queda de impureza** multiplicada pela **fração de amostras** que passaram por aquele nó:

$$\text{MDI}(X_j) = \frac{1}{B}\sum_{b=1}^{B} \sum_{\substack{\text{nós } t \text{ onde} \\ X_j \text{ foi usado}}} \underbrace{\Delta I(t)}_{\substack{\text{Limpeza} \\ \text{(Queda Gini)}}} \cdot \underbrace{\frac{N_t}{N}}_{\substack{\text{Peso do Nó} \\ \text{(\% da base)}}}$$

Regra de Bolso (`feature_importances_`)

- A soma de todos os atributos é normalizada para **100%** (1,0).
- **Posição importa:** Um corte na raiz (100% das amostras) vale 10× mais do que a mesma limpeza perto da folha (10% das amostras).



Exemplo: X_1 e X_2 tiveram a mesma queda $\Delta\text{Gini} = 0{,}30$, mas X_1 ganha 10× mais importância por atuar na raiz.

A Armadilha da MDI vs. O "Teste de Sabotagem" (Permutação)

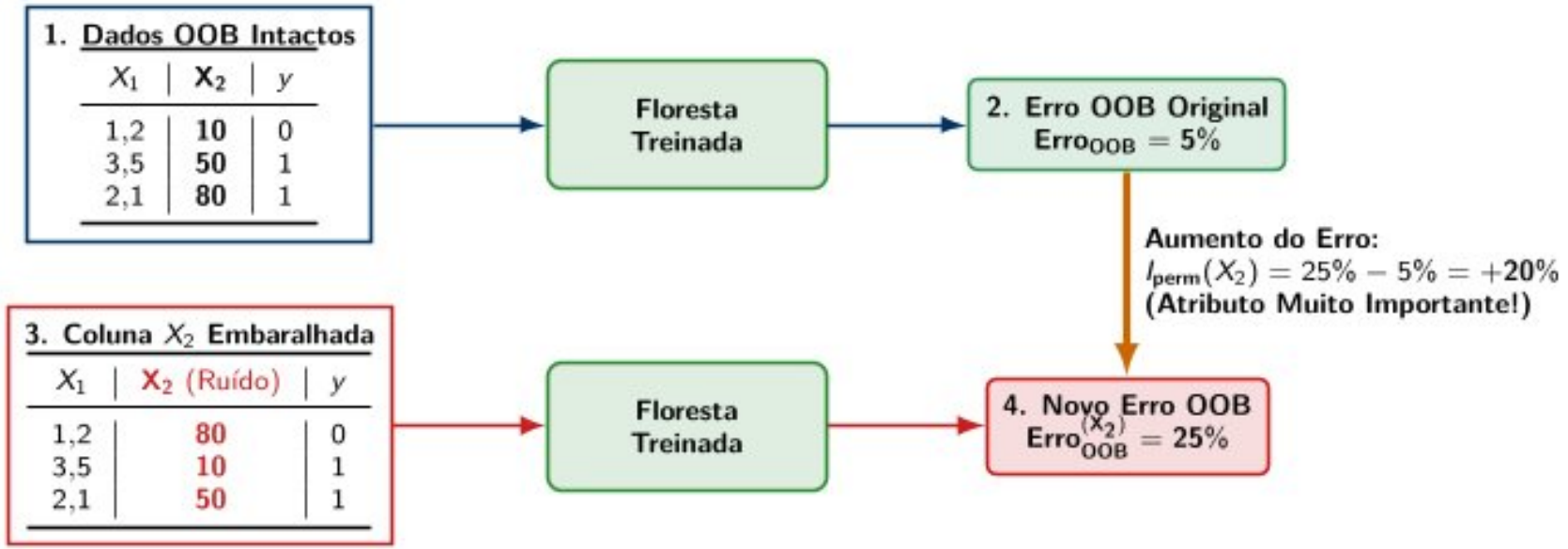
O Problema: O Vício da Redução Média de Impureza no Treino

- **Onde é medida:** Nos próprios dados de **treinamento** (*in-bag*).
- **A Armadilha da Alta Cardinalidade:** Variáveis com muitos valores distintos (ex: `CPF`, `CEP`, ou ruído contínuo) oferecem centenas de pontos de corte.
- **Consequência:** Nos nós profundos, a árvore usa essas variáveis para "decorar" exemplos isolados. A MDI soma essas pequenas limpezas de treino e acha que o ruído é importante!

A Solução: Importância por Permutação OOB

- **Onde é medida:** Nos dados de **validação Out-of-Bag** (que a árvore nunca viu).
- **A Lógica do "Teste de Sabotagem":** Como saber se o atributo X_j é realmente importante para o modelo?
- Nós **sabotamos** a coluna X_j (embaralhando seus valores entre os clientes OOB) e pedimos para a floresta prever novamente:
  - Se o erro da floresta **explodir**, X_j era importante!
  - Se o erro **não mudar**, X_j era inútil!

Funcionamento da Importância por Permutação OOB



Fórmula Geral da Importância por Permutação

$$\mathbf{I}_{perm}(\mathbf{X_j}) = Erro_{OOB}^{(j)} \text{ (após embaralhar } \mathbf{X_j}) - Erro_{OOB} \text{ (original)}$$

Exemplo Prático do Vício MDI: O "Atributo Ruído ID"

Considere adicionar uma coluna fictícia `ID_Cliente` com N números inteiros únicos $(1, 2, \ldots, N)$ a uma base de dados.

Comportamento do MDI (Gini Importance)

- Como `ID_Cliente` possui N valores distintos, o algoritmo encontra cortes aleatórios que separam pequenos grupos de treino.
- A MDI soma todas essas quedas espúrias de Gini no treino e atribui **importância** ao `ID_Cliente` fictício, enganando o cientista de dados!

Comportamento da Permutação OOB

- Ao embaralhar o `ID_Cliente` nas amostras OOB (não vistas no treino), o erro de generalização não muda (ou até melhora), pois a variável não contém informação preditiva real.
- A Permutação OOB (`permutation_importance`) atribui **importância** $\approx 0{,}0$ ao ruído!

Quadro Comparativo: Árvore Única vs. Ensembles

Modelo	Atributos no Nó	Seleção de Limiares	Principal Vantagem
Árvore Única	Todos os d	Ótimo (Busca Exaustiva)	Alta interpretabilidade visual.
Bagging	Todos os d	Ótimo (Busca Exaustiva)	Reduz variância de modelos instáveis.
Random Forest	Subconjunto $m < d$ ($m = \sqrt{d}$)	Ótimo (Busca Exaustiva)	Descorrelaciona árvores ($\rho \downarrow$) e previne overfitting.
Extra-Trees	Subconjunto $m < d$ ($m = \sqrt{d}$)	Totalmente Aleatório	Alta velocidade computacional e maior redução de variância.

Exemplo Numérico Passo a Passo: Dataset e Bootstrap

Considere um dataset de brinquedo com $N = 5$ amostras e $d = 2$ atributos para classificação binária $y \in \{0, 1\}$:

Tabela: Dataset Original $\mathcal{D}$

ID	X_1	X_2	**Alvo** y
A_1	1,0	2,0	0
A_2	2,0	1,0	0
A_3	3,0	4,0	1
A_4	4,0	3,0	1
A_5	5,0	5,0	1

Sorteio das $B = 3$ Amostras Bootstrap

- **Amostra $\mathcal{D}^{(1)}$ (para T_1):** $\{A_1, A_1, A_2, A_3, A_4\}$ $\Longrightarrow$ OOB : $\{\mathbf{A_5}\}$.
- **Amostra $\mathcal{D}^{(2)}$ (para T_2):** $\{A_2, A_3, A_4, A_5, A_5\}$ $\Longrightarrow$ OOB : $\{\mathbf{A_1}\}$.
- **Amostra $\mathcal{D}^{(3)}$ (para T_3):** $\{A_1, A_2, A_2, A_5, A_5\}$ $\Longrightarrow$ OOB : $\{\mathbf{A_3}, \mathbf{A_4}\}$.

Rastreando a Indução das 3 Árvores nas Amostras Bootstrap

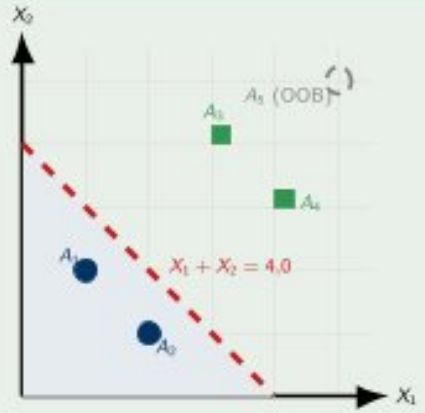
Como cada árvore enxerga um subconjunto diferente de exemplos e atributos sorteados, cada uma encontra um ponto de corte distinto para separar perfeitamente (Gini $= 0$) sua própria amostra bootstrap:

- **Árvore T_1 sobre $\mathcal{D}^{(1)} = \{A_1(1,2) \to 0,\ A_2(2,1) \to 0,\ A_3(3,4) \to 1,\ A_4(4,3) \to 1\}$:**
  - Avaliando a soma das coordenadas ($X_1 + X_2$): nos exemplos da Classe 0 (A_1, A_2), a soma vale **3**; nos exemplos da Classe 1 (A_3, A_4), a soma vale **7**. Qualquer limiar entre 3 e 7 separa as classes, definindo a regra $\mathbf{X_1 + X_2 \leq 4{,}0}$.
- **Árvore T_2 sobre $\mathcal{D}^{(2)} = \{A_2(2,1) \to 0,\ A_3(3,4) \to 1,\ A_4(4,3) \to 1,\ A_5(5,5) \to 1\}$:**
  - Sorteando o atributo X_1: o único exemplo da Classe 0 tem $X_1 = \mathbf{2{,}0}$ (A_2), e o menor exemplo da Classe 1 tem $X_1 = \mathbf{3{,}0}$ (A_3). O ponto médio entre 2,0 e 3,0 define a regra $\mathbf{X_1 \leq 2{,}5}$.
- **Árvore T_3 sobre $\mathcal{D}^{(3)} = \{A_1(1,2) \to 0,\ A_2(2,1) \to 0,\ A_5(5,5) \to 1\}$:**
  - Sorteando o atributo X_2: como A_3 e A_4 ficaram de fora (são OOB!), o maior valor de X_2 na Classe 0 é $\mathbf{2{,}0}$ (A_1) e o menor na Classe 1 é $\mathbf{5{,}0}$ (A_5). Escolhe-se um corte intermediário entre 2,0 e 5,0, definindo a regra $\mathbf{X_2 \leq 3{,}0}$.

Como Cada Amostra Bootstrap Gera um Corte Diferente?

Cada árvore enxerga apenas os pontos sorteados na sua amostra (pontos cheios) e ignora os pontos **OOB** (círculos tracejados), traçando uma fronteira diferente para separar a **Classe 0 (•)** da Classe 1 (■):

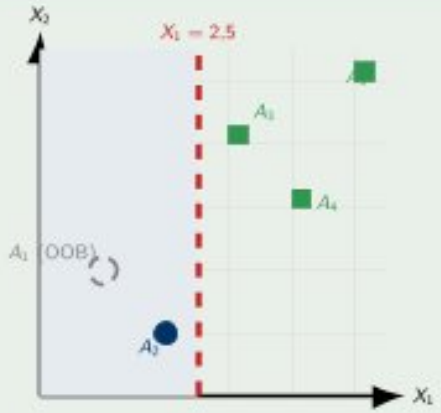
1. Árvore T_1 ($\mathcal{D}^{(1)}$ — OOB: A_5)



- **Padrão visto:** Na Classe 0 (A_1, A_2), a soma $X_1 + X_2 = \mathbf{3}$. Na Classe 1 (A_3, A_4), a soma vale **7**.
- **Corte escolhido:** Passa a reta diagonal entre 3 e 7:

$$\mathbf{X_1 + X_2 \leq 4{,}0}$$

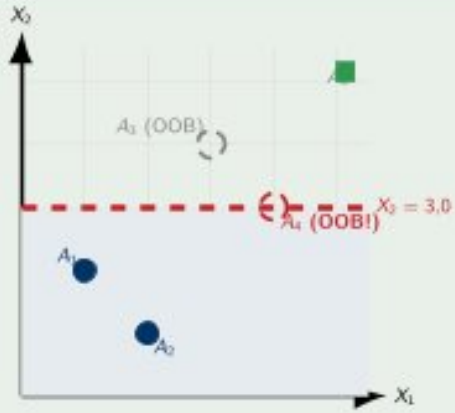
2. Árvore T_2 ($\mathcal{D}^{(2)}$ — OOB: A_1)



- **Atributo sorteado (X_1):** Maior X_1 da Classe 0 é A_2 ($X_1 = \mathbf{2{,}0}$); menor da Classe 1 é A_3 ($X_1 = \mathbf{3{,}0}$).
- **Corte escolhido:** Ponto médio exato entre 2,0 e 3,0:

$$\mathbf{X_1 \leq 2{,}5}$$

3. Árvore T_3 ($\mathcal{D}^{(3)}$ — OOB: A_3, A_4)



- **Atributo sorteado (X_2):** Sem ver A_3 e A_4, há um vazio entre A_1 ($X_2 = \mathbf{2{,}0}$) e A_5 ($X_2 = \mathbf{5{,}0}$).
- **Corte escolhido:** Traça a reta em 3,0 (que depois errará A_4 no OOB!):

$$\mathbf{X_2 \leq 3{,}0}$$

Estrutura e Regras de Corte das 3 Árvores

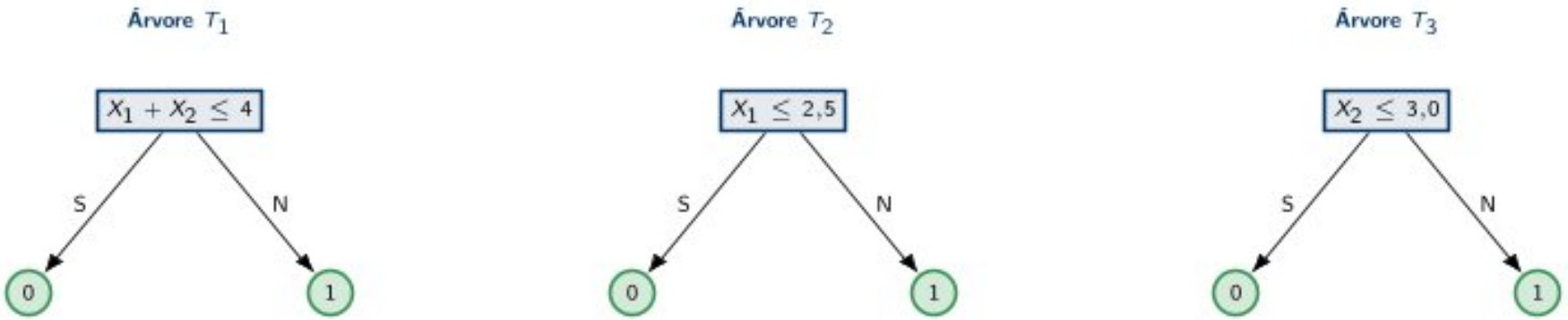


Figura: Regras de decisão aprendidas por cada árvore sobre sua amostra bootstrap.

Processo de Votação Majoritária no Teste

Dada a amostra de teste $\boldsymbol{x}_{\text{teste}} = (X_1 = 2{,}5, X_2 = 3{,}5)$:

Avaliação Individual Árvore a Árvore

- **Árvore T_1 ($X_1 + X_2 \leq 4{,}0$):** $2{,}5 + 3{,}5 = 6{,}0 > 4{,}0 \implies$ Ramo Não $\implies \widehat{\mathbf{y}}^{(1)} = \mathbf{1}$.
- **Árvore T_2 ($X_1 \leq 2{,}5$):** $2{,}5 \leq 2{,}5 \implies$ Ramo Sim $\implies \widehat{\mathbf{y}}^{(2)} = \mathbf{0}$.
- **Árvore T_3 ($X_2 \leq 3{,}0$):** $3{,}5 > 3{,}0 \implies$ Ramo Não $\implies \widehat{\mathbf{y}}^{(3)} = \mathbf{1}$.

Contagem de Votos e Decisão Final

- Votos para Classe $1 = 2$ (T_1, T_3).
- Votos para Classe $0 = 1$ (T_2).

$$\widehat{\mathbf{y}}_{\text{ens}}(\boldsymbol{x}_{\text{teste}}) = \arg\max\{\mathbf{1} : \mathbf{2}, \mathbf{0} : \mathbf{1}\} = \mathbf{1}$$

Matriz de Validação e Cálculo da Acurácia OOB

Avaliando a predição OOB **apenas sobre as árvores para as quais cada amostra foi OOB**:

Tabela: Matriz de Predições Out-of-Bag (OOB)

Amostra	**Alvo** y	**Árvores OOB**	**Predição Árvores OOB**	**Voto OOB**	**Acerto?**
$A_1(1,2)$	0	$\{T_2\}$	$T_2 : X_1 = 1 \leq 2{,}5 \rightarrow 0$	0	Sim
$A_2(2,1)$	0	Nenhuma	Participou de T_1, T_2, T_3	-	(Ignorada)
$A_3(3,4)$	1	$\{T_3\}$	$T_3 : X_2 = 4 > 3{,}0 \rightarrow 1$	1	Sim
$A_4(4,3)$	1	$\{T_3\}$	$T_3 : X_2 = 3 \leq 3{,}0 \rightarrow 0$	0	Não
$A_5(5,5)$	1	$\{T_1\}$	$T_1 : 5 + 5 = 10 > 4 \rightarrow 1$	1	Sim

Desempenho Global Out-of-Bag

Das 4 amostras avaliadas em OOB (A_1, A_3, A_4, A_5), o comitê acertou 3:

$$\text{Acc}_{\text{OOB}} = \frac{\mathbf{3}}{\mathbf{4}} = \mathbf{0{,}75} \quad (\mathbf{75{,}0\%})$$

Guia de Hiperparâmetros do `RandomForestClassifier`

Hiperparâmetro	Valor Padrão	Papel Matemático e Efeito no Modelo
`n_estimators`	`100`	Número de árvores B no comitê. Aumentar reduz a variância ($\frac{1-\rho}{B}\sigma^2 \to 0$) sem causar overfitting.
`max_features`	`'sqrt'`	Número de atributos m sorteados por nó ($m = \sqrt{d}$). Reduzir m diminui a correlação ρ entre árvores, mas eleva levemente o viés individual.
`bootstrap`	`True`	Ativa a amostragem com reposição. Se `False`, usa todo o dataset em cada árvore (desativando o OOB).
`oob_score`	`False`	Se `True`, calcula automaticamente a acurácia de validação nas 36,8% amostras Out-of-Bag (`rf.oob_score_`).
`max_depth` / `min_samples_leaf`	`None` / `1`	Controle de pré-poda. Por padrão, as árvores crescem até folhas puras (baixo viés), deixando o controle da variância para a votação.
`n_jobs`	`None`	Como as árvores são independentes, `n_jobs=-1` paraleliza o treinamento em todos os núcleos da CPU.

Referências Bibliográficas

- **GÉRON, Aurélien.** *Mãos à Obra: Aprendizado de Máquina com Scikit-Learn, Keras & TensorFlow*. 2. ed. Alta Books, 2021.
- **FACELI, K. et al.** *Inteligência Artificial: Uma Abordagem de Aprendizado de Máquina*. 2. ed. LTC, 2021.
- **MORETTIN, P. A.; SINGER, J. M.** *Estatística e Ciência de Dados*. LTC, 2022.
- **LUGER, George F.** *Inteligência Artificial*. 6. ed. Pearson, 2013.